

README

1. 实验背景与目标

本次实验旨在深入分析模型推理的性能瓶颈，并探索有效的加速方案。

主要目标：

- 诊断 (Profiling)**: 使用性能分析工具定位推理过程中的主要耗时点。
- 优化 (Optimization)**: 针对识别出的瓶颈实施代码层面的优化。
- 验证 (Verification)**: 通过量化指标对比优化前后的性能差异。

2. 性能瓶颈分析 (Profiling)

在进行优化尝试之前，我首先使用 `torch.profiler` 对基线模型进行了详尽的性能分析。

2.1 基线性能数据

在 WikiText-2 数据集上进行的初步测试显示：

- 吞吐量 (Throughput)**: 约 60.41 tokens/s，性能有待提升。
- 单步解码延迟 (TPOT)**: 约 16.55 ms，影响实时交互体验。

2.2 瓶颈定位

通过分析 Profiling 生成的 Trace 视图，我发现了一个显著的问题：**CPU 瓶颈 (CPU-Bound)**。

- 在解码阶段，CPU 耗时高达 **77ms**，而 GPU 计算仅需 **40ms**。
- 这意味着 GPU 有近一半的时间处于空闲等待状态 (GPU Starvation)，未能发挥出应有的计算能力。

进一步检查代码发现，原生的 Attention 实现由大量细粒度的算子 (MatMul, Scale, Mask, Softmax 等) 组成。这些操作虽然逻辑清晰，但导致了频繁的 Python 解释器开销和 CUDA Kernel Launch 开销，同时增加了显存带宽的压力。

结论：主要瓶颈在于算子过于碎片化导致的 CPU 调度开销，而非 GPU 算力不足。

3. 优化方案实施

基于上述分析，我决定采用 **算子融合** 技术来解决 CPU 调度瓶颈。具体采用 **Flash Attention** 机制进行优化。

3.1 核心改动

我通过继承 `GPTNeoXAttention` 类并重写其 `forward` 函数来实现优化。不再依赖原有的手动矩阵运算，而是直接调用 PyTorch 提供的优化算子 `F.scaled_dot_product_attention`。

代码对比示意：

代码块

```
1  # [优化前] 手动实现 Attention, 产生大量中间变量和 Kernel 开销
2  # attn_scores = torch.matmul(query, key.transpose(-1, -2)) / self.norm_factor
3  # attn_scores = attn_scores + attention_mask
4  # attn_probs = nn.functional.softmax(attn_scores, dim=-1)
5  # ...
6
7  # [优化后] 使用 Flash Attention, 算子融合, 减少显存访问
8  with torch.backends.cuda.sdp_kernel(enable_flash=True, enable_math=True,
   enable_mem_efficient=True):
9      output = F.scaled_dot_product_attention(
10         query, key, value,
11         attn_mask=None,
12         dropout_p=self.attention_dropout if self.training else 0.0,
13         is_causal=True
14     )
```

该方法利用 Flash Attention 的分块计算策略，将计算主要保持在 GPU 的 SRAM 中，显著减少了对 HBM 的访问次数。

4. 实验结果验证

优化完成后，我重新进行了 Benchmark 测试，并与基线数据进行了对比。

4.1 性能指标对比

指标	基线模型 (Baseline)	优化后模型 (Optimized)	提升幅度	说明
TPOT (解码延迟)	16.55 ms	9.96 ms	降低 39.8%	成功消除了 CPU 调度瓶颈。
Throughput (吞吐量)	60.41 tokens/s	96.01 tokens/s	提升 1.59倍	推理效率显著提升。
FLOPs (算力利用率)	0.34 TFLOPS	0.54 TFLOPS	提升 1.59倍	GPU 算力得到更充分利用。
TTFT (首字延迟)	23.97 ms	22.84 ms	持平	Prefill 阶段主要受计算限制，优化空间较小。

4.2 复测 Profiling

再次运行 Profiler 显示，优化后的 CPU 耗时已降至约 20ms，完全被 GPU 计算时间 (约 40ms) 所覆盖。这表明我们成功将系统瓶颈从 CPU 转移回了 GPU，达到了硬件利用率的上限。

5. 环境配置与复现指南

为保证实验的可复现性，以下记录了实验环境配置及操作步骤。

5.1 实验环境

- OS: Linux
- GPU: NVIDIA GPU (推荐 8GB+ 显存)
- Python: 3.10
- Dependencies: PyTorch 2.0+ (支持 SDPA), Transformers, Accelerate

5.2 复现步骤

1. 安装依赖:

代码块

```
1 pip install torch transformers accelerate modelscope datasets addict
```

1. 运行 Benchmark: 执行以下命令自动下载模型并运行性能测试:

代码块

```
1 python benchmark_2_8b.py
```

1. 运行 Profiling: 如需深入查看耗时分布，可运行：

代码块

```
1 python profile_2_8b.py
```

6. 实验总结

本次实验通过引入 Flash Attention 技术，成功解决了 Pythia-2.8B 模型在推理过程中的 CPU 瓶颈问题。实验结果表明，在不升级硬件的前提下，仅通过代码层面的算子融合优化，即可将推理吞吐量提升近 60%。这也验证了在大模型推理工程实践中，“先诊断后优化”这一方法论的重要性。